

EDU-CORE

Educational Digital Processor ASIC

Detailed Design & Module Implementation Specification

Parameter	Value
Foundry	SCL Chandigarh, 180nm CMOS
Target Node	SCL 180nm CMOS (1.8V core / 3.3V IO), 4M1IL metal stack
Standard-cell library	tsl18fs120_scl (fs120, 4M1IL) — ff / ss corners
I/O pad library	tsl18cio150 (4M1L pad frame)
Reference project flow	Cadence Genus (synthesis) → Innovus (P&R) → Calibre/Assura (signoff)
Class of design	Fully synchronous digital logic, zero SRAM macros, zero analog PLL
Status	Specification complete — ready for RTL coding

This document specifies EDU-CORE, a minimal RISC-V-inspired educational processor whose 16-instruction set implements fundamental digital-logic building blocks (adder/subtractor, gate array, encoder, decoder, multiplexer, shifter/rotator, two's-complement, binary/Gray converter, and parity generator/checker) rather than conventional ALU arithmetic. The design is intended to be carried through a full RTL-to-signoff ASIC flow on the SCL 180nm PDK, using the same project standard (directory layout, pad-frame convention, and library set) already established by the reference tapeout package supplied for this project.

This specification follows the documentation conventions of two reference documents supplied for style guidance: an executive/architecture-pitch style tapeout proposal, and a per-module implementation specification with full port tables, behaviour tables, and verification checklists. Every module needed to build EDU-CORE is documented to the same depth.

1 Executive Summary & Tapeout Overview

EDU-CORE is a fully hardwired, switch-and-LED operable digital logic teaching processor. A student enters a 16-bit instruction word on 16 DIP switches, presses **Load**, and the chip decodes the opcode, routes the operands to the selected functional block, and displays the 9-bit result (8-bit value + 1 carry/flag bit) on LEDs. Because every functional block is a small, purely synchronous/combinational digital circuit with no memory macros or clock-tree PLLs, the design carries very low tapeout risk on SCL 180nm — the same risk profile as the reference FFT project this specification's directory and pad-frame conventions are drawn from.

1.1 Primary Silicon & Architecture Specification

Parameter	Specification	Architectural Justification / Impact
Process / Foundry	SCL 180nm CMOS (4M1IL)	tsl18fs120_scl standard cells + tsl18cio150 I/O pads
Operating Voltage	1.8V core / 3.3V I/O	Dual-rail split power domains; LVCMOS-compatible switches/LEDs
Instruction Width	16 bits (opcode/rd1/rd2/mem_addr, 4b each)	Directly matches the 16-DIP-switch physical panel
Datapath Width	4-bit register operands, 8-bit internal operand (paired nibbles), 9-bit output	Matches 8-LED + 1-flag-LED output panel
Instruction Count	16 opcodes (one reserved for memory write)	Fully decodable with a 4-bit opcode field
Execution Model	Single-cycle capture + combinational execute	Result is valid one clock after Load is asserted
Memory	16 x 4-bit read/write register file	Addressable via the 4-bit mem_addr field
Clock	External, free-running (board oscillator or manual step button)	Keeps the design fully synchronous and STA-clean
Reset	Active-high, asynchronous	Simple pushbutton reset, matches reference project's rst convention
Silicon class	Zero SRAM macros, zero analog PLL, pure synchronous digital	Minimizes fabrication risk; matches reference project's yield argument

1.2 Strategic Tapeout Defense Points

- **Guaranteed silicon yield:** pure synchronous digital datapath, zero memory macros or analog PLL dependency, exactly as in the reference FFT tapeout — the same class of design that has already been proven through this flow.
- **Complete testability:** every input maps directly to a DIP switch and every output to an LED; no hidden internal state that cannot be observed from the package pins.
- **Wide timing margin:** the deepest combinational path (4-bit ripple add/subtract, 3-bit priority encode, or an 8-bit shift) is a handful of gate delays in 180nm — large slack is expected against any reasonable clock target (see §7).
- **Distinct educational value:** unlike a conventional ALU-only core, EDU-CORE lets a student physically operate an encoder, decoder, multiplexer, code converter, and parity checker — the fundamental building blocks taught in a first digital-logic course — on real silicon.

2 Technology & Pad-Ring Specification

EDU-CORE reuses the exact library and flow standard already established by the reference tapeout project (32-point FFT) supplied alongside this specification: same standard-cell library, same I/O pad library, same synthesis/place-and-route/signoff tool chain, and the same `rtl / synthesis / layout / signoff / work` directory layout.

Item	Value
Standard-cell library	tsl18fs120_scl (fs120, 4M1IL) — corners: _ff.lib, _ss.lib
I/O pad library	tsl18cio150 (4M1L pad frame) — corners: cio150_max.lib, cio150_min.lib
LEF	scl18fs120_tech.lef, scl18fs120_std.lef, tsl18cio150_4lm.lef, tsl180i4.lef
Parasitics	generated_captable_4M.cap, qrcTechFile_4M
Synthesis	Cadence Genus (syn_gen → syn_map → syn_opt)
Place & route	Cadence Innovus (place → CTS → route → STA → signoff)
Project tree	makeProjectTree ... -flow SCLSL18_4M1L -cds -attach -ver -assura -pvs

2.1 Top-Level Pin List

Pin	Dir	Width	Pad Cell	Description
in[15:0]	Input	16	pc3d01 x16	16-bit DIP-switch instruction word
rst	Input	1	pc3d01	Active-high asynchronous reset (pushbutton)
load	Input	1	pc3d01	Load strobe (DIP switch / pushbutton), sampled synchronously
clk	Input	1	pc3d01 + pc3c04	System clock (external oscillator or manual step button)
out[8:0]	Output	9	pc3o01 x9	out[7:0] = result, out[8] = carry / borrow / flag
VDD / VSS	Power	—	pvd0c / pv0c	Core power (digital logic)
VDDO / VSSO	Power	—	pvd0a / pv0a	I/O buffer AC-section power
VDD / VSS (predriver)	Power	—	pvd0i / pv0i	Core + I/O predriver power
VSS (short)	Power	—	pv0f	Ties VSS / VSSO busses together
Corner cells	—	4	pfrelr	Maintains continuous VDD/VSS metal rings at corners

Total digital I/O: **28 signal pins** (19 in, 9 out). Together with power/corner pads this is comparable in scale to the reference project's pad ring (34 signal pins), so the same small-pad-count padframe floorplan strategy applies directly. A standard 40/44-pin QFN or DIP package comfortably accommodates the full pad count.

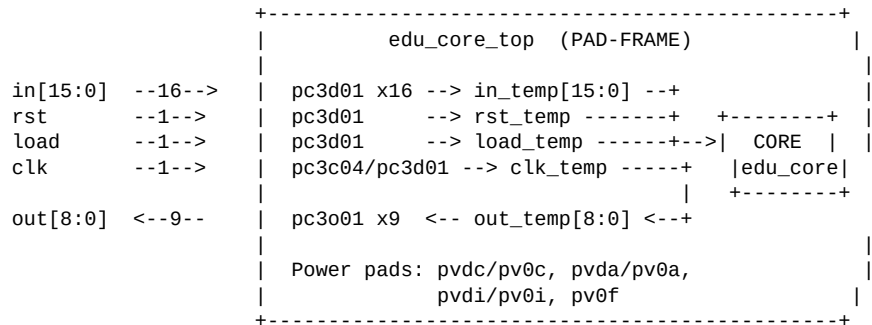
2.2 DIP-Switch Bit Mapping

Switch #	1	2	3	4		5	6	7	8		9	10	11	12		13	14	15	16
Bit	in15	in14	in13	in12		in11	in10	in9	in8		in7	in6	in5	in4		in3	in2	in1	in0
Field	----- opcode -----					----- rd1 -----					----- rd2 -----					----- mem_addr -----			

Switches read **left** → **right** = **MSB** → **LSB** across the whole 16-bit word; within each 4-bit field the rightmost switch of that group is the field's LSB. This order should be silk-screened on the physical panel to match the RTL bit order exactly.

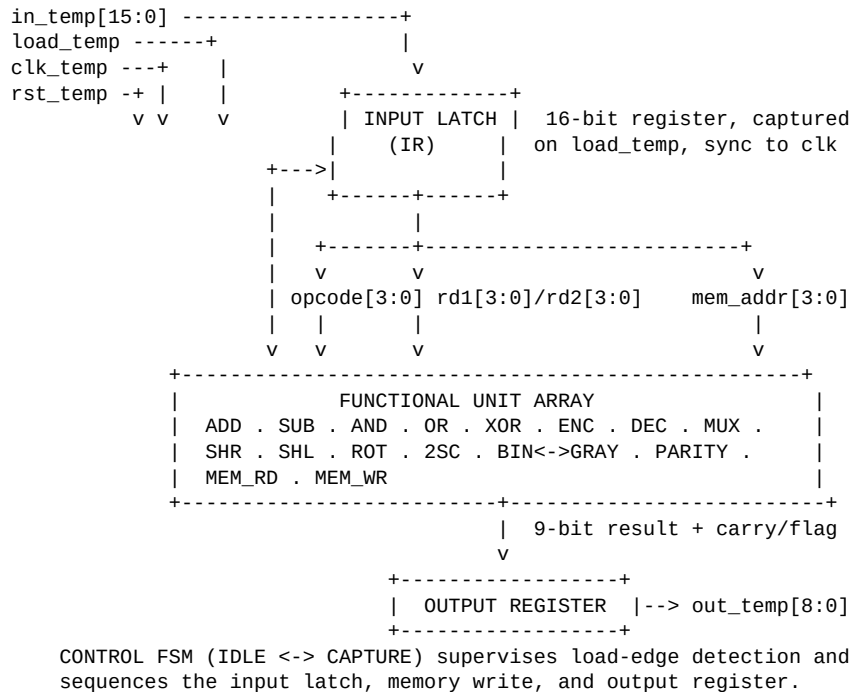
3 System Architecture

3.1 Top-Level Block Diagram (Pad-Frame Wrapper)



edu_core_top is the pad-ring wrapper (structurally identical in role to `top.v` in the reference project): it instantiates I/O pad cells from `tsl18cio150` around the digital core and carries no functional logic itself. **edu_core** is the synthesizable digital core (equivalent role to `topFFT` in the reference project).

3.2 Core-Level Block Diagram



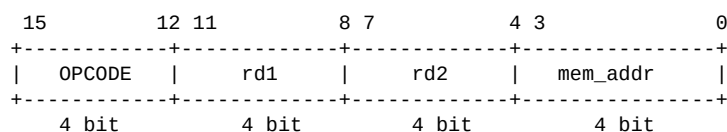
3.3 Module List

Module	Type	Role
edu_core_top	Structural (pad-frame)	Instantiates I/O pads around edu_core; no logic
edu_core	Top of synthesizable core	Wires input_latch, control_fsm, functional_unit_array, mem16x4, output_register together
input_latch	Sequential	Captures in[15:0] into IR on a detected load edge

Module	Type	Role
control_fsm	Sequential	2-state IDLE/CAPTURE controller; edge-detects load, drives write-enables
functional_unit_array	Combinational	Implements all 16 opcodes' datapath logic from IR fields
mem16x4	Sequential (write) / Combinational (read)	16 x 4-bit read/write register file
output_register	Sequential	Registers the 9-bit result onto out[8:0]

4 Instruction Set Architecture Reference

All instructions are 16 bits wide with a fixed 4-field format:



Field	Bits	Description
opcode	[15:12]	Operation code — selects the instruction (16 possible)
rd1	[11:8]	Operand / register 1
rd2	[7:4]	Operand / register 2
mem_addr	[3:0]	Memory address (used by MEM_RD / MEM_WR)

Output word format — out [8:0]:

Bits	Meaning
[7:0]	Result of the selected operation
[8]	Carry / borrow / flag bit (meaning depends on instruction — see §6)

4.1 Complete Instruction Set — All 16 Opcodes

Opcode	Hex	Mnemonic	Operation	Description
0000	0x0	MEM_RD	out[3:0] <- mem[mem_addr]	Read memory
0001	0x1	ADD	{out[8],out[3:0]} <- rd1+rd2	4-bit add, carry-out on out[8]
0010	0x2	SUB	out[3:0] <- rd1-rd2	4-bit subtract, borrow-out on out[8]
0011	0x3	AND	out[3:0] <- rd1 & rd2	Bitwise AND
0100	0x4	OR	out[3:0] <- rd1 rd2	Bitwise OR
0101	0x5	XOR	out[3:0] <- rd1 ^ rd2	Bitwise XOR
0110	0x6	ENC	out[2:0] <- encode({rd1,rd2})	8-to-3 priority encoder
0111	0x7	DEC	out[7:0] <- decode(rd1[2:0])	3-to-8 decoder
1000	0x8	MUX	out[0] <- rd1[rd2[1:0]]	4x1 multiplexer
1001	0x9	SHR	out[7:0] <- {rd1,rd2} >> 1	Logical shift right by 1
1010	0xA	SHL	out[7:0] <- {rd1,rd2} << 1	Logical shift left by 1
1011	0xB	ROT	out[7:0] <- rotr({rd1,rd2},1)	Rotate right by 1
1100	0xC	TWOSCOMP	out[7:0] <- ~{rd1,rd2}+1	2's complement
1101	0xD	CONV	dual-mode (see §6)	Binary <-> Gray conversion
1110	0xE	PARITY	dual-mode (see §6)	Even-parity generate / check
1111	0xF	MEM_WR	mem[mem_addr] <- rd2[3:0]	Write memory (was reserved — see §9.4)

5 Module: input_latch

Module	Type	Status
input_latch	Sequential	TODO — you must implement

Purpose: Captures the 16-bit instruction word from the pad-frame into an internal instruction register (IR) on a detected Load event, so that the rest of the core sees a stable, glitch-free instruction for the entire following clock cycle.

```

clk      ----->+-----+
rst      ----->|         |
in[15:0] ----->|   input_latch   |-----> ir[15:0]
load_pulse ----->|         |
              +-----+
(pposedge clk / rst is asynchronous)

```

Port	Direction	Width	Description
clk	input	1	System clock — IR updates on rising edge
rst	input	1	Active-high asynchronous reset — IR immediately clears to 16'h0000
in	input	16	Raw 16-bit word from the DIP-switch pads
load_pulse	input	1	Single-cycle load strobe from control_fsm (already edge-detected)
ir	output	16	Registered, stable instruction word for this execution cycle
Condition		Action	
rst = 1		ir <- 16'h0000 (asynchronous, overrides everything)	
rst = 0, load_pulse = 1		ir <- in[15:0] on the next rising edge of clk	
rst = 0, load_pulse = 0		ir holds its current value	

Implementation checklist: declare a 16-bit reg; use always @(posedge clk or posedge rst); place the reset branch first; use non-blocking assignment (<=); never latch combinationally.

6 Module: control_fsm

Module	Type	Status
control_fsm	Sequential	TODO — you must implement

Purpose: Supervises the Load handshake. Converts a possibly-long DIP-switch/pushbutton assertion of Load into a single one-cycle pulse (avoiding multiple captures from one press), and generates the write-enable pulses for the input latch, the memory write port, and the output register.

Port	Direction	Width	Description
clk	input	1	System clock
rst	input	1	Active-high asynchronous reset
load	input	1	Raw load input from the DIP switch / pushbutton pad
opcode	input	4	Current opcode field of the captured instruction (from IR)
load_pulse	output	1	Single-cycle pulse: capture the instruction into IR this cycle
mem_wr_en	output	1	Single-cycle pulse: write mem[mem_addr] <- rd2 (opcode = MEM_WR only)
out_wr_en	output	1	Single-cycle pulse: register the functional-unit result into out[8:0]

```

+-----+   load==1 (rising edge, synchronized)   +-----+
rst->| IDLE | ----->| CAPTURE |
+-----+ <-----+
      ^               next clk: latch in->IR, decode+execute
      |               (combinational), register result,
+-----+-----write memory if opcode==MEM_WR-----+

```

Implementation checklist: add a 2-flip-flop synchronizer + rising-edge detector on load to produce load_pulse; out_wr_en follows load_pulse one cycle later (once the functional unit has settled); mem_wr_en = load_pulse & (opcode == 4'hF). This is the only module in the design that needs to reason about instruction timing — every functional unit itself is purely combinational.

7 Module: functional_unit_array

Module	Type	Status
functional_unit_array	Combinational	TODO — you must implement

Purpose: The computational heart of EDU-CORE. Takes the latched opcode, rd1, rd2, and the current memory read data, and computes the 9-bit result for every one of the 16 instructions. Purely combinational — no clock, mirroring the ALU module style of a conventional educational core, but switching on 16 digital-logic operations instead of 16 arithmetic operations.

Port	Direction	Width	Description
opcode	input	4	Operation select, from IR[15:12]
rd1	input	4	Operand 1, from IR[11:8]
rd2	input	4	Operand 2, from IR[7:4]
mem_rdata	input	4	Current value of mem[mem_addr], for MEM_RD
result	output	9	result[7:0] = operation output, result[8] = carry/borrow/flag

7.1 Per-Instruction Operand Mapping & Behaviour

0000 — MEM_RD

- result[3:0] = mem_rdata (4-bit read value, zero-extended); result[7:4] = 0; result[8] = 0.

0001 — ADD

- {result[8], result[3:0]} = rd1 + rd2 (5-bit sum; result[8] = carry-out); result[7:4] = 0.

0010 — SUB (borrow on result[8])

- Computed as {cout, diff} = rd1 + (~rd2 + 1); result[8] = ~cout (borrow-out, active-high: 1 when rd1 < rd2).
- result[3:0] = diff; result[7:4] = 0.

0011 / 0100 / 0101 — AND / OR / XOR

- 4-bit bitwise operation on rd1, rd2 -> result[3:0]; result[7:4] = 0; result[8] = 0.
- Any subset of switches can be used as single- or multi-bit gates simply by driving only the desired switch positions.

0110 — ENC (8-to-3 priority encoder, MSB-first priority)

- 8-bit input = {rd1, rd2}. result[2:0] = index of the highest-order '1' bit (bit 7 = highest priority); result[7:3] = 0.
- result[8] = valid flag: 1 if at least one input bit is 1, else 0 (defined behaviour for the all-zero case).

0111 — DEC (3-to-8 decoder)

- 3-bit select = rd1[2:0]. result[7:0] = one-hot (bit <select> = 1, all others 0); result[8] = 0.

1000 — MUX (4x1)

- Data inputs: the four individual bits rd1[3], rd1[2], rd1[1], rd1[0]. Select: rd2[1:0]. rd2[3:2] are don't-cares.
- result[0] = rd1[rd2[1:0]]; result[8:1] = 0.

1001 / 1010 — SHR / SHL (fixed 1-bit logical shift)

- 8-bit operand = {rd1, rd2}.
- SHR: result[7:0] = {1'b0, operand[7:1]} (zero-fill at MSB). SHL: result[7:0] = {operand[6:0], 1'b0} (zero-fill at LSB).
- result[8] = the bit shifted out (SHR: operand[0]; SHL: operand[7]), reported as a flag.

1011 — ROT (rotate right by 1)

- 8-bit operand = {rd1, rd2}. result[7:0] = {operand[0], operand[7:1]}; result[8] = 0.

1100 — TWOSCOMP

- 8-bit operand = {rd1, rd2} (rd1 = MSB nibble, rd2 = LSB nibble). result[7:0] = ~operand + 1.
- result[8] = overflow flag, set to 1 only when operand == 8'h00 (the defined edge case where 2's complement of 0 has no true negative), else 0.

1101 — CONV (Binary <-> Gray, dual-mode)

- If rd1 != 0 and rd2 == 0: rd1 treated as 4-bit binary -> result[3:0] = binary_to_gray(rd1).
- If rd2 != 0 and rd1 == 0: rd2 treated as 4-bit Gray -> result[3:0] = gray_to_binary(rd2).
- If both non-zero (or both zero): invalid -> result[7:0] = 0, result[8] = 1 (error flag, so the invalid case is visibly distinct from a legitimate all-zero result).

1110 — PARITY (dual-mode)

- Generate (rd1 != 0, rd2 == 0): p = XOR of rd1[3:0] (even parity bit). result[4:0] = {p, rd1[3:0]}; result[7:5] = 0.
- Check (rd2 != 0, rd1 == 0): rd2[3:0] is a 4-bit codeword {p_in, d[2:0]}. result[0] = 1 if (p_in ^ d[2] ^ d[1] ^ d[0]) != 0 (parity error detected), else 0; result[7:1] = 0.
- Both non-zero or both zero: invalid, same convention as CONV -> result[7:0] = 0, result[8] = 1.

1111 — MEM_WR

- Not decoded inside functional_unit_array itself — handled by control_fsm asserting mem_wr_en and mem16x4 capturing rd2[3:0] into mem[mem_addr]. result output for this opcode is don't-care (recommend result = 9'b0 for a clean LED display while the write completes).

Implementation checklist: use a single always @(*) block with a case (opcode) statement; set result = 9'b0 as the default at the top of the block (prevents inferred latches); implement all 16 cases explicitly, including MEM_WR returning 0.

8 Module: mem16x4

Module	Type	Status
mem16x4	Sequential (write) / Combinational (read)	TODO — you must implement

Purpose: 16-entry x 4-bit read/write memory, addressed by mem_addr. Read is asynchronous (combinational, feeds functional_unit_array within the same cycle); write is synchronous, gated by mem_wr_en from control_fsm.

Port	Direction	Width	Description
clk	input	1	System clock — write happens on posedge clk
rst	input	1	Active-high asynchronous reset — clears all 16 words to 4'h0
wr_en	input	1	Write enable, asserted only when opcode = MEM_WR (from control_fsm)
addr	input	4	Address — shared by read and write ports (mem_addr field of IR)
wr_data	input	4	Data to write — rd2[3:0] of IR
rd_data	output	4	Combinational read output: mem[addr]
Condition		Action	
rst = 1		All 16 words <- 4'h0 (asynchronous)	
rst = 0, wr_en = 1		mem[addr] <- wr_data, on the next rising edge of clk	
rst = 0, wr_en = 0		Memory contents hold	
(any time)		rd_data = mem[addr] (combinational, no clock latency)	

Implementation checklist: declare `reg [3:0] mem [0:15]`; read with a single assign `rd_data = mem[addr]`; write inside `always @(posedge clk or posedge rst)` using non-blocking assignment; on reset, clear the array with a for-loop. At this size (64 bits total) this infers as a small distributed register array in synthesis — no dedicated SRAM macro is needed, keeping the design fully synthesizable and macro-free like the reference project.

9 Module: output_register

Module	Type	Status
output_register	Sequential	TODO — you must implement

Purpose: Registers the combinational result from functional_unit_array onto the output pads, so the LEDs display a clean, glitch-free value that holds steady until the next Load event (rather than flickering combinational while the DIP switches are still being set).

Port	Direction	Width	Description
clk	input	1	System clock
rst	input	1	Active-high asynchronous reset — clears output to 9'b0
out_wr_en	input	1	Single-cycle pulse from control_fsm: capture result this cycle
result_in	input	9	Combinational result from functional_unit_array
out	output	9	Registered value driving the output pads (out[7:0] + out[8])

Implementation checklist: a plain 9-bit always @(posedge clk or posedge rst) register, gated by out_wr_en; no combinational logic here.

9.1 edu_core Top-of-Core Wiring (structural only, no new logic)

```

module edu_core (
    input  clk, rst, load,
    input  [15:0] in,
    output [8:0] out
);
    wire [15:0] ir;
    wire      load_pulse, mem_wr_en, out_wr_en;
    wire [3:0] mem_rdata;
    wire [8:0] result;

    control_fsm CTRL (
        .clk(clk), .rst(rst), .load(load), .opcode(ir[15:12]),
        .load_pulse(load_pulse), .mem_wr_en(mem_wr_en), .out_wr_en(out_wr_en) );

    input_latch IL (
        .clk(clk), .rst(rst), .in(in), .load_pulse(load_pulse), .ir(ir) );

    mem16x4 MEM (
        .clk(clk), .rst(rst), .wr_en(mem_wr_en),
        .addr(ir[3:0]), .wr_data(ir[7:4]), .rd_data(mem_rdata) );

    functional_unit_array FUA (
        .opcode(ir[15:12]), .rd1(ir[11:8]), .rd2(ir[7:4]),
        .mem_rdata(mem_rdata), .result(result) );

    output_register OREG (
        .clk(clk), .rst(rst), .out_wr_en(out_wr_en),
        .result_in(result), .out(out) );
endmodule

```

10 Golden Test Vectors & Verification

The vectors below are hand-computed from the behaviour defined in §7 and should be used as the self-checking reference in `testbench.v`, mirroring the golden-vector verification approach used for the reference project's own accelerator core.

#	Opcode	rd1	rd2 / mem_addr	Expected out[8:0]	Meaning
1	ADD	1001 (9)	1000 (8)	1_0001	9+8=17, carry=1, result=0001
2	SUB	0011 (3)	0101 (5)	1_1110	3-5=-2, borrow=1, result=1110 (14)
3	AND	1010	0110	0_0010	0xA & 0x6 = 0x2
4	OR	1010	0110	0_1110	0xA 0x6 = 0xE
5	XOR	1010	0110	0_1100	0xA ^ 0x6 = 0xC
6	ENC	0010	0000	1_0000101	bit5 set -> index 5, valid=1
7	DEC	1011	—	0_00001000	rd1[2:0]=3 -> one-hot bit3
8	MUX	1010	0011	0_00000001	select=3 -> rd1[3]=1
9	SHR	1011	0100	0_01011010	{0xB,0x4}=0xB4 >> 1 = 0x5A
10	TWOSCOMP	0000	0001	0_11111111	~1+1 = 0xFF (-1); overflow flag=0
11	CONV (bin→gray)	0110	0000	0_0101	binary 0110 -> gray 0101
12	CONV (gray→bin)	0000	0111	0_0101	gray 0111 -> binary 0101
13	PARITY (gen)	1010	0000	0_01010	data 1010, even parity p=0
14	PARITY (chk, error)	0000	1011	0_00000001	codeword 1,011: parity fails
15	PARITY (chk, ok)	0000	0110	0_00000000	codeword 0,110: parity holds
16	MEM_WR then MEM_RD	—	rd2=0111, addr=0101	0_00000111	write 7 to addr 5, then read it back

10.1 Module-Level Test Checklist

Test	Module(s) Under Test	What to Check
Reset	All	rst=1 -> IR=0, memory=0, out=0
ADD / SUB	functional_unit_array	Carry and borrow flags on out[8] for boundary values (0+0, 15+15, 0-15)
AND / OR / XOR	functional_unit_array	Known bit patterns (0xA, 0x5, 0xF)
ENC all-zero input	functional_unit_array	valid flag (out[8]) correctly reports 0
DEC	functional_unit_array	One-hot output for all 8 select values
MUX	functional_unit_array	All 4 select values route the correct rd1 bit
SHR / SHL / ROT	functional_unit_array	Shifted-out bit correctly reported on out[8]
TWOSCOMP of zero	functional_unit_array	Overflow flag = 1 only for operand = 0
CONV both non-zero	functional_unit_array	Invalid case -> out=0, error flag=1
PARITY both non-zero	functional_unit_array	Invalid case -> out=0, error flag=1

Test	Module(s) Under Test	What to Check
MEM_WR -> MEM_RD	mem16x4, functional_unit_array	Data written is read back correctly at the same address
Load debounce	control_fsm	A single long Load press captures exactly once

11 Static Timing & Synthesis Setup

EDU-CORE's deepest combinational path is a 4-bit ripple add/subtract, an 8-bit priority encode, or an 8-bit shift/rotate — all a few gate levels in 180nm. A conservative clock target is proposed to give very large slack, since the chip's real-world pace is set by a student toggling switches, not by clock frequency.

Parameter	Value
Target clock frequency	10 MHz (nominal); functionally correct well beyond this given the shallow logic depth
Clock period constraint	100 ns
Clock slew / uncertainty	Rise/fall = 1.0 ns; setup uncertainty = 1.0 ns; hold uncertainty = 0.4 ns
I/O timing budget	Input max delay = 20 ns (20%); output max delay = 20 ns (20%); pin load = 5 pF
Synthesis path groups	IN2REG, REG2REG, REG2OUT, IN2OUT (segmented, matching reference project convention)
Expected critical path	Comfortably under 5 ns pre-layout — >90% slack against the 100 ns constraint

12 Design Hierarchy & File/Directory Plan

Following the same project layout as the reference tapeout package:

```

edu_core_workshop/
|-- rtl/
|   |-- edu_core_top.v          -- pad-frame wrapper (like top.v)
|   |-- edu_core.v             -- core wiring (like topFFT)
|   |-- input_latch.v
|   |-- control_fsm.v
|   |-- functional_unit_array.v
|   |-- mem16x4.v
|   |-- output_register.v
|   |-- testbench.v
|   |-- filelist_GLS
|-- synthesis/
|   |-- synthesis.tcl          -- Genus script (mirrors reference synthesis.tcl)
|   |-- constraints.sdc
|   |-- filelist_GLS
|-- layout/
|   |-- config.tcl / place.tcl / cts.tcl / route.tcl / sta_route.tcl / signoff.tcl
|   |-- top_netlist.io        -- pad-frame floorplan order
|-- signoff/
|   |-- testbench_GLS.v
|   |-- pdk_flow_setup.csh
|   |-- filelist_GLS
`-- work/

```

The RTL_LIST for synthesis.tcl (analogous to the reference project's RTL_LIST) will be: edu_core.v input_latch.v control_fsm.v functional_unit_array.v mem16x4.v output_register.v, with DESIGN = "edu_core" (the core only is synthesized; edu_core_top is stitched in at the layout stage, exactly as topFFT was synthesized separately from top.v in the reference flow).

12.1 Tapeout Readiness & Repository Artifacts

Artifact / File	Format & Type	Design Phase / CAD Tool	Target Status
edu_core.v + submodules	Synthesizable Verilog RTL	Architecture / Genus	Zero lint warnings
edu_core_top.v	SCL 180nm pad-frame top netlist	Full-chip top / Innovus	All pads bound
top_netlist.io	Innovus floorplan IO file	Physical design floorplan	Clockwise pad order, matches §2.1
constraints.sdc	Synopsys SDC	STA signoff (Genus / Innovus)	10 MHz / 100 ns period / 5 pF load
testbench.v	Self-checking testbench	Simulation (Icarus / Xcelium)	100% golden-vector match (§10)

13 Consolidated Design-Decision Log

The original informal specification left a number of points ambiguous. Each has been closed out with a specific, implementable decision so RTL can be written directly from this document.

#	Topic	Decision
1	Clock	Added a dedicated clk input; fully synchronous design
2	DIP-switch order	Left-to-right = MSB-to-LSB across the 16-bit word
3	SUB flag	out[8] = borrow-out, active-high
4	MUX data inputs	The 4 individual bits of rd1
5	Shift/rotate amount	Fixed at 1 bit per operation
6	Rotate direction	Rotate right
7	Opcode 1111	Assigned to MEM_WR (previously reserved/unused)
8	CONV / PARITY both-nonzero case	Invalid -> out=0, out[8]=1 error flag
9	Memory write path	Only via MEM_WR (opcode 1111); data = rd2
10	TWOSCOMP of zero	out[8] overflow flag set only when operand = 8'h00

Next steps: RTL coding of each module in §5–§9 following the exact port lists and behaviour tables given above; the synthesis script and layout/signoff templates then follow the reference project's existing scripts with the RTL_LIST and DESIGN name given in §12.